PROJECT 4.2 & 4.3 · UNIT 4 — CONTROL & COMPETITION

Sumo Competition & Build Your Own

Student Edition — VEX EXP · C++ in VS Code

Name: _____ **Date:** _____ **Period:** _____

Introduction

The finale — a **build**, not a lecture. Everything from this year comes down to two capstone challenges. First, a head-to-head **robot sumo** tournament: design a robot that pushes its opponent out of the ring. Then **build your own** — pick a real problem and design a robot to solve it, autonomous or driver-controlled. There's no new command to learn; instead you'll combine **building**, **gears** for pushing power, **sensors** and autonomous decisions, and **driver control**, run through the **engineering design process** you've used since day one. Read Chapter 14 in the textbook for the full picture.

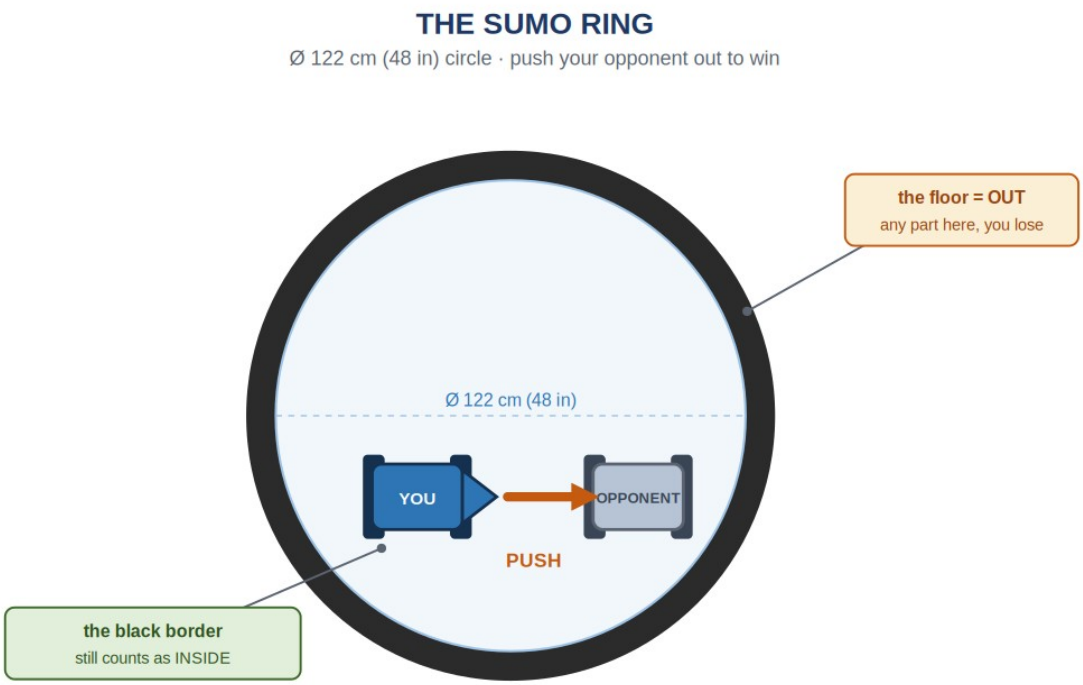
Learning Objectives

- Design, build, and program a robot for a rules-based competition (robot sumo) and for an open-ended problem you choose.
- Explain where pushing power comes from — torque (gear down), traction, and a low, wide base — connecting it to Chapters 3, 4, and 7.
- Choose between an autonomous routine (Chapter 11) and driver control (Chapter 13), and justify the choice.
- Run the full engineering design process and use a decision matrix to pick a design by reasons, not by opinion.

Key Ideas (quick reference)

- **Sumo, in one line:** two robots, one ring — push the other robot out. A robot is OUT when any part touches the floor outside the ring (the black border still counts as IN). Matches are 2 minutes, double elimination.
- **Pushing power = torque + traction + staying low:** gear DOWN to trade speed for torque (torque is pushing force, Ch 4); get traction from grippy wheels with weight over them (spinning wheels go nowhere); build LOW and WIDE so you're hard to tip or shove (Ch 7). A wedge that lifts the opponent's wheels steals their grip. Speed does not win sumo — force does.
- **Drive it or automate it:** most teams DRIVE (tank drive, Ch 13 — you react faster than a simple program). You CAN automate with a Distance Sensor that finds and charges the opponent (Ch 11); staying in the ring then needs edge sensing (a downward Optical Sensor on the black border).
- **The design process + the decision matrix:** define → sketch → choose → build → test → improve. For the CHOOSE step, score each idea against your criteria (1–4, 4 = best) and total the columns — the highest total is your best design. You pick by reasons, not by whoever argues loudest.

Challenge 1: Robot Sumo (Honbasho)

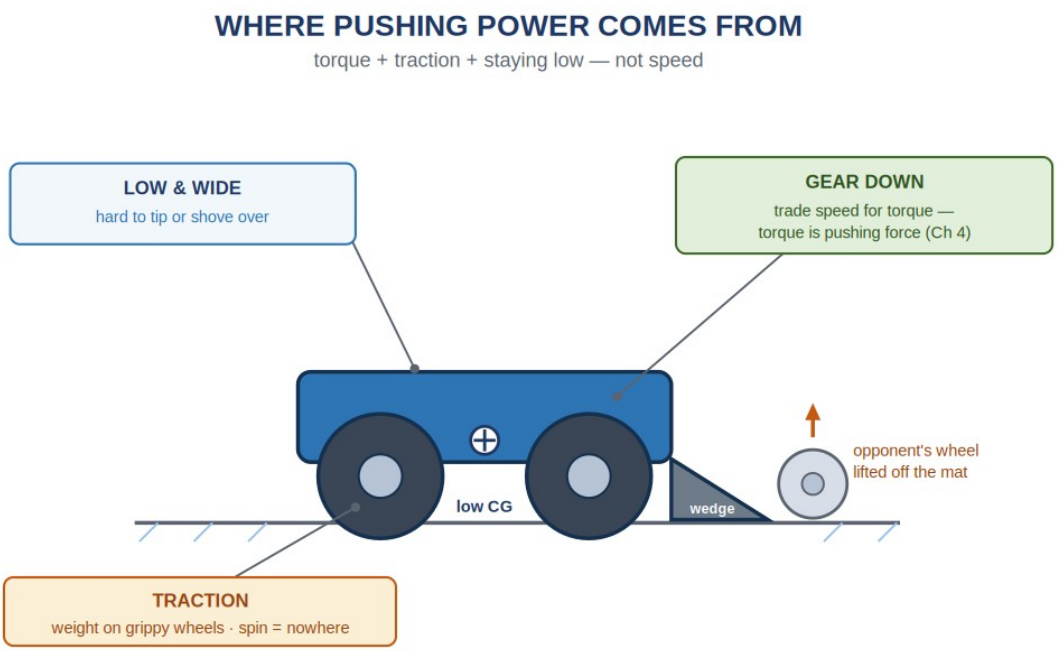


Robot sumo on a 122 cm (48 in) ring. Push your opponent out — or get them to drive themselves out. Any part touching the floor outside the black border is out; the border itself still counts as in.

Two robots, one ring, one job: **push the other robot out**. It's king of the hill on wheels. We use a simplified version of the official Japanese rules:

- A match is 2 minutes; the tournament is double elimination (out after two losses).
- An inspection before each match checks the rules and safety — an unsafe robot doesn't compete.
- Start on the referee's go; jumping the start costs a freeze penalty. Once it starts, you can't touch your robot.
- A robot is OUT when any part touches the floor outside the ring (even a broken-off piece). You WIN by pushing your opponent out, or if they drive themselves out.
- 15 seconds stopped = giving up = a loss. If no one is out at time, judges decide on technical merit.
- No interference (signal jamming) and no deliberate damage — it's a pushing contest, not a demolition derby.

Strategy: Where Pushing Power Comes From



Pushing power is torque + traction + a low, wide stance — not top speed. A wedge that slides under the opponent lifts their wheels off the mat and steals their grip.

Winning sumo is an engineering problem, and you already know the engineering. The robot that **pushes harder** and **stays planted** wins — and that comes down to **torque** (gear down; torque is pushing force, Ch 4), **traction** (grippy wheels with weight over them — spinning wheels go nowhere), and a **low, wide base** (hard to tip or shove, Ch 7). A **wedge** at the front that slips under the opponent and lifts their wheels off the mat is the single most effective add-on.

Drive It, or Let It Drive Itself?

This is the Chapter 11 vs. Chapter 13 choice. Most sumo teams **drive** (you react faster than a simple program) — the tank drive from Chapter 13. But you can make sumo **autonomous**, using the Distance Sensor from Chapter 11 to find and charge the opponent:

A simple autonomous sumo — find the opponent, then charge

```
distance frontEye  = distance(PORT2);   // find the opponent
motor leftMotor  = motor(PORT1);
motor rightMotor = motor(PORT6, true);

int main() {
    vexcodeInit();
    while (true) {
        // opponent close ahead? charge.  else, spin to search:
        if (frontEye.objectDistance(mm) < 400) {
            leftMotor.spin(forward);
            rightMotor.spin(forward);
        } else {
            leftMotor.spin(forward);
            rightMotor.spin(reverse);
        }
        wait(20, msec);
    }
}
```

It's the same autonomous shape from Chapter 11: a forever loop, a sensor reading, and an if that decides. It hunts (spin until the Distance Sensor sees something close), then charges. What it

doesn't solve is staying in the ring — for that a robot senses the **edge**, usually with a downward Optical Sensor that spots the black border and backs away (a great stretch goal).

Challenge 2: Build Your Own

The brief is one sentence: **find a problem, and build a robot to solve it** — autonomous or driver-controlled, your choice. Same constraints (kit only, fits the cubby); everything else, including what counts as **success**, is yours. Pick a problem that's real but small enough to build:

- A sorting or delivery bot that moves an object, or pushes objects into the right bin.
- A cleanup bot that plows scattered objects into a corner.
- A maze or line follower that navigates with sensors.
- A guard bot that watches a doorway and reacts when the Distance Sensor sees something approach.
- A drawing or stamping bot, or a fetch bot that grabs and returns an object.

Write *what the robot must do* and *how you'll know it worked* before you build — that sentence is your design statement and your finish line.

Your Toolkit: the Decision Matrix

Both challenges run on the **engineering design process**: define → sketch → choose → build → test → improve (a loop — testing sends you back). For the **choose** step, use a **decision matrix**: list your criteria down the side and your ideas across the top, score each idea 1–4 (4 = best), and total the columns. The highest total is your best starting design — chosen by reasons, not by argument. Here's one for a sumo robot:

Criteria (what matters)	A: wedge	B: fast	C: heavy
Pushing power (torque)	4	2	3
Stays in the ring (low & wide)	3	2	4
Easy to build from the kit	4	3	2
Fits the cubby	4	4	3
Total	15	11	12

Idea A wins — not because it's anyone's favorite, but because it scores best against the criteria the team agreed on. You make your own criteria from your design brief.

Build Constraints

> **Build constraints (still in force).** (1) Build from the classroom PLTW Gateway kit only — its bundled motors, controller, and sensors are fair game; the separate competition team's V5 parts are off-limits. (2) The whole robot must fit one IKEA Kallax cubby — a 33 × 33 cm (13 × 13 in) opening, about 38 cm (15 in) deep. (3) For sumo: the robot must fit a 30 cm (12 in) square footprint (which also fits the cubby) and be well under about 2.3 kg (5 lb) — easy with kit parts. No part may be built to damage the other robot or the ring, and no signal interference.

Safety

- Robots collide and move fast in a match. Keep hands, hair, faces, and cables well clear of the ring while robots are running — watch from outside.

- The inspection is a safety check, not a formality: nothing sharp, nothing that throws parts, nothing that could hurt a person or wreck the kit.
- Only the driver stays near the field during a match; everyone else steps back.
- Unplug the USB-C cable before you compete. Power the Brain down with the X button and store your robot in its cubby when you're done.

Equipment & Materials

- Your computer with VS Code + the VEX extension, and a USB-C cable
- The PLTW Gateway EXP kit: Brain, charged battery, the controller, motors, and any sensors your design uses
- Your engineering notebook for sketches, the decision matrix, pseudocode, and a daily journal
- This project worksheet (design brief, decision matrix, build & code plan, journal, and reflection)
- For Build Your Own: any extra materials your robot needs to do its job (your team brings these)

Procedure

1. Define the problem. Write a short design brief and every constraint. For sumo the constraints are given; for Build Your Own, you write the problem and how you'll know it worked.
2. Sketch and choose. Each teammate sketches a couple of ideas, then build a decision matrix, score the ideas against your criteria, and pick the winner by reasons.
3. Build it. Construct the robot from the kit so it fits the cubby (and, for sumo, the footprint). Build it sturdy; mount and wire the motors and any sensors to the Brain.
4. Plan, then code. Decide autonomous or driver control. Write pseudocode first, then translate to C++: a while (true) loop with sensor-driven if's (Ch 11) or stick-and-button driving (Ch 13).
5. Test and refine — a little at a time. Fix one thing per round: more torque if it gets pushed, a wider stance if it tips, a flipped motor if a side runs backward.
6. Evaluate and present. Keep your daily journal, judge how well your robot met its goal, and present your design and process to the class.

Worksheet 1 — Design Brief

Define the problem before you build. For sumo the constraints are given; for Build Your Own, you write the problem and the finish line.

Section	Your answer
Which challenge? (Sumo / Build Your Own)	
Client / who needs this robot	
Problem statement — what is the need?	
Design statement — what it must do (and how you'll know it worked)	
Constraints (kit only, fits cubby; sumo: 30 cm footprint)	
Control method (autonomous or driver-controlled?)	

Section	Your answer

Worksheet 2 — Decision Matrix

Write your own criteria down the left (from your design brief), label your ideas A–D across the top, score each 1–4 (**4 = best**), and add each column. The highest **Total** is your best design.

Criteria (write your own)	A	B	C	D	Total
Total					

Worksheet 3 — Build & Code Plan

Sketch your chosen design in your notebook, then write its logic in plain language — **before** you write C++. Keep the half you're not using as a reference.

Pseudocode

```
/*
    devices: motors  (+ a sensor if autonomous)

    while (true)
    {
        // AUTONOMOUS — sense, then decide:
        if ( ____________________ )        // a sensor reading
            ____________________           // do this
        else
            ____________________           // otherwise

        // -- or DRIVER CONTROL — drive from the sticks --
        left  speed = _________________     (left stick)
        right speed = _________________     (right stick)
        if ( __________ ) _____________     // a button

        wait(20, msec)
    }
*/
```

Worksheet 4 — Daily Journal

A couple of sentences per day — especially what got in your way and how you fixed it.

Day	What we did / obstacles we hit and how we solved them
1	
2	
3	
4	

Conclusion Questions

Answer in complete sentences.

1. In robot sumo, how do you win a match, and what makes a robot out of the ring?
2. Why won't the fastest robot necessarily win sumo? Explain where pushing power comes from, using the word torque.
3. Name two things that make a robot hard to push out of the ring, and connect each to something you learned earlier in the course.
4. What is a decision matrix, and why is it a better way to choose a design than just picking the idea you like best?
5. If you ran your robot fully autonomously (no driver), what sensor(s) would it need and what would each one do? (Think about finding the opponent and staying in the ring.)
6. Describe two malfunctions your team had to troubleshoot, and how you overcame them.
7. Describe two responsibilities you held during this project.
8. Suggest one idea for a future robotics competition your class could run. (Remember: no destroying robots allowed.)